Repository security and AI architecture review
Repository reviewed: vitaliibalint39-oss/ai-powered-soc-lab
Commit: 9be444ff7c271711d90920dbdac2bf62e3bbf382
Method: Static review of the committed source, documentation, configuration, dependencies, examples, and screenshots. I did not execute the script against a live Wazuh or LM Studio installation.

Executive assessment

This is a credible, honestly scoped proof of concept, not yet a production-capable or continuously operating SOC component.
The strongest parts are:
Wazuh alert access is designed to go through an SSH tunnel rather than exposing OpenSearch.
TLS certificate verification is enabled instead of using verify=False.
The project explicitly requires analyst review.
The model is not given tools or the ability to execute remediation.
Credentials, reports, processed state, SSH keys, and the local CA are excluded by .gitignore.
The documentation clearly states that this is manual batch processing.
The principal blockers are:
The ingestion design can permanently miss alerts.
Committed screenshots disclose unnecessary operational and personal identifiers.
One screenshot contradicts the claimed loopback-only LM Studio configuration.
AI output has no structured contract, validation, deterministic fallback, or human-review workflow.
The claimed least-privilege OpenSearch role is not reproducible from the repository.
There are no automated tests or CI, despite a test plan containing mostly unimplemented verification items.

Actual architecture

The repository contains one executable Python module, one runtime dependency, documentation, one synthetic event example, one sanitized report example, and screenshots.
The real data flow is:
Text

```
Wazuh Agent
   → Wazuh Manager / OpenSearch Indexer
   → SSH local port forward
   → soc_assistant.py on Windows
   → LM Studio OpenAI-compatible endpoint
   → local Markdown reports
   → processed_events.json
```

The implementation is in https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assistant.py.
There is currently:
No daemon or scheduler.
No queue.
No API service.
No alert correlation engine.
No case-management integration.
No analyst approval state machine.
No structured LLM output contract.
No automated test suite.
Calling this an “AI-powered SOC lab” is acceptable as a learning project. It should not yet be described as a functioning SOC pipeline.

Prioritized findings

P0 — Critical

1. Alert ingestion can permanently skip events
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assistant.py#L42-L68 requests only the latest 20 matching documents:
Python

```
payload = {
    "size": BATCH_SIZE,
    ...
}
```

It then filters against previously processed IDs. If more than 20 new matching alerts arrive between runs, the oldest unprocessed alerts can fall outside the latest 20 and will never be returned again. This is not merely a performance limitation; it is an event-loss defect in a security pipeline.
The README correctly acknowledges this limitation, but the implementation should not proceed past proof-of-concept status without correcting it.

Required design:
Use search_after, not only document IDs.
Add a deterministic secondary sort such as _id.
Store a per-agent checkpoint.
Paginate through all matching events.
Persist state transactionally.
Distinguish fetched, analyzed, failed, and human_review_required.
2. Public screenshots expose personal and network information
Several screenshots should be removed, cropped, or replaced with synthetic images:
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/06-wazuh-agent-status.png exposes the Linux username and command history.
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/07-python-ai-analysis.png exposes the Windows profile path beginning with [REDACTED] , internal host naming, event content, and previous report paths.
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/08-qwen-api-server.png shows LM Studio reachable at a non-loopback address: http://[REDACTED] :1234.
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/01-wazuh-endpoint-dashboard.png and https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/04-security-configuration-assessment.png publish internal addresses, browser security status, and the monitored host’s security posture.
The internal IPs are not credentials, but the combination of usernames, hostnames, network topology, defensive tooling, and host weaknesses unnecessarily increases exposure.
Because these files are already committed, deleting them in a new commit does not remove them from Git history. If the owner considers the published details sensitive, history should be rewritten and any affected credentials or configuration rotated where applicable.
3. LM Studio is shown bound beyond localhost
The code and documentation default to:
Python
```
LM_STUDIO = "http://127.0.0.1:1234/v1"
```
However, the committed LM Studio screenshot shows the server as reachable at http://[REDACTED] :1234. That may be a WSL or host virtual-network address, but it is not loopback.
The same screenshot displays LM Studio developer endpoints, including model inference and model-download operations. The repository provides no evidence of API authentication or network restrictions for that listener. If that binding is still active, other code reachable on the same interface may be able to consume the inference API or cause resource consumption. Even if the exposure is limited to the local virtual network, the documentation should not imply a strictly localhost-only deployment.
Required controls:
Bind LM Studio to 127.0.0.1 or ::1.
Add a startup validation that rejects non-loopback inference endpoints by default.
If remote inference is deliberately supported, require authentication and encryption.
Verify listeners with OS network tooling.
Remove the public screenshot or replace it with a cropped, loopback-only image.

P1 — High

4. AI output is accepted without a structured safety contract
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assistant.py#L80-L103 accepts any non-empty model response:
Python
```
answer = (r.json()["choices"][0]["message"].get("content") or "").strip()
if not answer:
    raise RuntimeError(...)
return answer
```
There is no:
JSON schema.
Severity enum.
Confidence range validation.
Observed-facts/hypothesis separation at the data layer.
Required human-review flag.
Hallucination check.
Retry with structured output.
Fail-closed disposition.
Model or prompt version recorded in the report.
The system prompt asks the model to separate facts from hypotheses and avoid invented details, but that is

guidance, not a control. A malformed but non-empty hallucination is currently accepted and persisted.
The immediate impact is limited because the model has no tools and cannot execute commands. The larger risk is analyst overreliance on an unvalidated narrative.
Recommended contract:
JSON
```
{
  "event_id": "string",
  "disposition": "BENIGN | SUSPICIOUS | MALICIOUS | INSUFFICIENT_EVIDENCE",
  "severity": "INFO | LOW | MEDIUM | HIGH | CRITICAL",
  "confidence": 0.0,
  "observed_facts": ["Only claims directly supported by the event"],
  "hypotheses": ["Explicitly labeled speculation"],
  "recommended_readonly_steps": ["Safe analyst checks only"],
  "requires_human_review": true,
  "model_id": "string",
  "prompt_version": "string"
}
```
Validate it with Pydantic. If parsing or validation fails, classify as INSUFFICIENT_EVIDENCE and route to human review rather than saving the output as a completed analysis.
5. The model can silently change
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assistant.py#L71-L77 silently falls back to the first available model when the expected Qwen model is absent:
Python
```
return next(
    (m["id"] for m in models if "qwen3.5-4b-instruct" in m["id"].lower()),
    models[0]["id"]
)
```
That means behavior, accuracy, token limits, and safety characteristics can change without warning.
This should fail closed. Load an explicitly configured model or verify that exactly one approved model identifier is available.
6. Prompt-injection protection is currently only instructional
The code sends raw event fields to the model:
Python
```
{"role": "user", "content": json.dumps(alert, ensure_ascii=False)}
```
JSON encoding preserves the data structure, but it is not a security boundary. A log message can contain text such as “ignore previous instructions.” The current defense is a system prompt telling the model not to follow it.
That is better than nothing, but insufficient as an AI-security control.
Improvements:
Use a strict field allowlist.
Separate event fields from instructions with explicit delimiters.
Avoid passing unnecessary raw fields such as full_log when structured fields are enough.
Request structured JSON.
Validate all output.
Do not expose tools, shell access, network actions, or credential access to the model.
Add prompt-injection regression fixtures.
7. The least-privilege claim is not reproducible
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/docs/SECURITY.md says soc_reader is scoped to wazuh-alerts-*, but the repository contains no:
OpenSearch role definition.
User creation commands.
Permission mapping.
Negative authorization tests.
Wazuh or Indexer configuration as code.
TLS/certificate setup script.
A single observed HTTP 403 is useful evidence, as the document notes, but it is not proof of least privilege across all write and administrative paths.
Provide the exact role JSON and automated tests proving that the account can search allowed indexes but cannot write documents, modify security configuration, access unrelated indexes, or perform destructive index operations.
8. State and report handling are not transactional
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assi

stant.py#L35-L38 uses a temporary file and os.replace(), which is good, but it does not prevent concurrent runs or align report writes with state changes.
In https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/soc_assistant.py#L123-L142:
The model output is generated.
The report is written directly.
The event ID is added to memory.
The whole state file is rewritten.
Failure or overlap can result in:
Duplicate reports.
Lost state updates from concurrent processes.
Partial report files.
Reports written but state not updated.
State updated despite incomplete report persistence.
Use SQLite or another small transactional store with:
event_id as a primary key.
Report status.
Retry count.
Last error.
An advisory lock or single-worker ownership.
Atomic report creation through a temporary file followed by os.replace().
9. There are no automated tests or CI
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/docs/TEST_PLAN.md is useful, but important items remain unverified or unimplemented:
Repeat-run behavior.
LM Studio outage behavior.
SSH tunnel failure behavior.
TLS validation failure.
Alert volumes above 20.
Redaction before publication.
The repository has no tests/ directory and no GitHub Actions workflow.
At minimum, add mocked tests for:
OpenSearch pagination and checkpoint behavior.
Duplicate event IDs.
Equal timestamps.
Model outage.
Empty model output.
Malformed model output.
Non-approved model selection.
Report write failure.
State corruption.
Concurrent execution.
Non-loopback LM Studio endpoint rejection.
10. Dependency management is minimal
https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/requirements.txt contains:
Text
requests>=2.31,<3
This is a reasonable runtime range, but there is:
No lockfile.
No cryptographic hash pinning.
No dependency vulnerability scanning.
No SBOM.
No CI enforcement.
I am not claiming a known vulnerability in the dependency range. The issue is that the repository has no mechanism to detect or reproduce a safe dependency set.
Use pip-tools or uv, commit a lockfile, generate hashes, and run pip-audit in CI.

P2 — Medium

11. Alert filtering and processing are not operationally configurable
The following are hard-coded:
Python
INDEXER = "https://127.0.0.1:9200"

LM_STUDIO = "http://127.0.0.1:1234/v1"
USERNAME = "soc_reader"
BATCH_SIZE = 20
MIN_LEVEL = 5
In addition, agent.name is hard-coded inside the query as ubuntu-soc.
Add typed configuration through environment variables or a config file, plus a checked-in .env.example. The .gitignore already anticipates an example file, but none exists.
12. Reporting and state retention are underspecified
Reports include event IDs, timestamps, agent names, rule descriptions, and unrestricted model output. The event passed to the model can include data.srcip and full_log.
The repository correctly warns that reports can contain sensitive information, but it does not provide:
Automatic redaction.
Data-retention rules.
Report permissions.
A publishing/sanitization tool.
An analyst approval step.
If reports remain Markdown, sanitize or redact before sharing and define how long local reports and processed IDs are retained.
13. Minor code defects and maintainability issues
Specific issues in the current implementation:
The initial ascending sort is overwritten by a descending sort at lines 44 and 59. This is harmless but stale and confusing.
safe_event = dict(event) at line 127 is unused. Worse, its name implies redaction that is not actually performed.
datetime.now() uses local time rather than UTC.
REPORTS.mkdir(exist_ok=True) does not set restrictive permissions.
State elements are not validated as strings; malformed mixed-type state may fail during sorted(ids).
Errors are printed to stdout/stderr rather than logged with levels, timestamps, and structured fields.
There is no retry/backoff policy for transient network failures.
The state file is rewritten in full after every event.

Documentation findings

Positive
The documentation is unusually clear about what the project does not do. It explicitly says:
Manual batch processing only.
No autonomous incident response.
Model output can be wrong.
Reports may contain sensitive information.
The synthetic SSH event is not a real attack.
That scope discipline is valuable.
Defects
Clone/path mismatch: The README and Ukrainian setup guide tell the user to run:
powershell
cd "$HOME\AI-SOC-Lab"
A normal clone of this repository will be named ai-powered-soc-lab. Either document the rename step or use the actual repository name.
README rendering: The architecture fence does not specify mermaid, and the final screenshot headings appear concatenated without line breaks.
Screenshot evidence is inconsistent with the current source: https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/07-python-ai-analysis.png displays analysis of a Wazuh level 3 event, while the current script filters for level 5 or higher. It also shows older wazuh_ai_report_*.txt output, while the current code writes incident_*.md. This may be older output, but it should be labeled or regenerated from the committed version.
Screenshot guide omission: https://github.com/vitaliibalint39-oss/ai-powered-soc-lab/blob/9be444ff7c271711d90920dbdac2bf62e3bbf382/screenshots/SCREENSHOT_GUIDE.md does not include the committed 08-qwen-api-server.png in its table.
No license: There is no LICENSE file. Absence of a license does not make the project freely reusable by default. Add an explicit license if public reuse is intended.
Security policy location: docs/SECURITY.md contains project security notes, but there is no root-level vulnerability-reporting policy.

Security architecture review

Controls that should be retained
These are good design decisions:

OpenSearch access through SSH local forwarding.
session.verify = str(CA_FILE) rather than disabling certificate validation.
Runtime password collection with getpass.
Concept of an unprivileged soc_reader.
No automated blocking or command execution by the LLM.
Clear separation between synthetic evidence and real incidents.
.gitignore exclusions for reports, state, root CA material, private keys, virtual environments, and logs.
Explicit warning against committing credentials and raw incident data.
Remaining security gaps
Public artifacts disclose more environment detail than necessary.
LM Studio network binding is not validated or restricted in code.
Least-privilege configuration is not committed or independently testable.
No automated secret scanning or pre-commit checks.
No report redaction or approval workflow.
No data-retention policy.
No dependency or CI security gates.
Dashboard TLS trust is not fully documented; screenshots show the browser treating the dashboard connection as "Not secure."
The lab host shown in screenshots has substantial Wazuh vulnerability inventory counts and low CIS assessment scores. Those counts do not prove every finding is exploitable, but they should either be remediated or explicitly explained as part of an intentionally unhardened lab.

Recommended target AI architecture

The next iteration should treat the LLM as an untrusted advisory component inside a deterministic control plane.
Recommended processing pipeline
Text

```
OpenSearch
  → validated alert fetch
  → idempotency/checkpoint store
  → allowlisted event normalization
  → prompt-injection-resistant structured prompt
  → approved local model
  → schema-validated JSON response
  → deterministic policy checks
  → human-review queue
  → atomic report persistence
```

AI controls to add
Approved-model allowlist
Do not silently fall back to another model.
Structured output only
Use JSON mode where available and validate with Pydantic.
Fail closed
Malformed, incomplete, contradictory, or low-confidence output becomes INSUFFICIENT_EVIDENCE and requires human review.
No tool execution
Keep the initial version read-only.
Prompt-injection tests
Include hostile full_log values in the test suite.
Reproducibility metadata
Store model ID, prompt version, temperature, schema version, and analysis timestamp.
Calibrated confidence handling
Treat model confidence as uncalibrated unless evaluated. Do not let it independently trigger high-impact conclusions.
Human disposition tracking
Persist analyst outcomes: confirmed, benign, duplicate, insufficient evidence, escalated.

Prioritized development roadmap

Phase 0 — Immediate containment: 1–2 days
Remove or replace the public screenshots with synthetic or heavily redacted versions.
If sensitive exposure warrants it, rewrite Git history rather than merely deleting the files.
Verify LM Studio is bound to loopback.
Fix the README clone path and rendering defects.
Add a LICENSE.
Add root SECURITY.md with a vulnerability-reporting process.

Add .env.example.
Acceptance criteria:
No usernames, profile paths, internal IPs, or unredacted event content appear in committed media.
No non-loopback model listener is active by default.
A fresh clone can follow the README without guessing the project directory.
Phase 1 — Reliable ingestion and state: 1–2 weeks
Replace the 20-alert window with search_after pagination.
Sort by timestamp plus a unique tie-breaker.
Store per-agent checkpoints.
Replace JSON state with SQLite or an equivalent transactional store.
Add an execution lock.
Write reports atomically.
Add retries, structured logging, and failure states.
Add a scheduler or service wrapper.
Acceptance criteria:
1,000 synthetic matching events are processed with zero omissions and zero duplicates.
Repeated runs are idempotent.
Killing the process after any step cannot corrupt state.
More than 20 new alerts between runs are all processed.
Phase 2 — AI safety and output contracts: 2–3 weeks
Define a versioned Pydantic response schema.
Remove silent model fallback.
Add loopback endpoint validation.
Add structured prompt-injection defenses.
Record model and prompt metadata.
Route malformed output to human review.
Add an analyst disposition workflow.
Add redaction for publishable report samples.
Acceptance criteria:
100% of saved AI analyses conform to the schema.
Malformed model responses never produce completed reports.
A non-approved model causes a clear failure.
Hostile text inside full_log cannot alter the output schema or trigger tool behavior.
Phase 3 — Engineering quality and supply chain: 2–3 weeks
Split the script into typed modules:
configuration
OpenSearch client
state store
LLM client
report writer
Add CLI arguments.
Add pytest coverage with mocked OpenSearch and LM Studio servers.
Add Ruff or Flake8, Black, mypy, Bandit, and pip-audit.
Add GitHub Actions.
Pin dependencies and commit a lockfile.
Generate an SBOM if this will be used beyond a personal lab.
Acceptance criteria:
CI runs on every pull request.
Dependency scanning is enforced.
Core ingestion and AI-contract tests run without live infrastructure.
Code coverage measures the checkpoint, failure, and schema-validation paths.
Phase 4 — Reproducible security infrastructure: 3–4 weeks
Commit the soc_reader OpenSearch role definition.
Add scripts or infrastructure-as-code for user and role creation.
Document Wazuh Indexer TLS and hostname/SAN requirements.
Add negative authorization tests for write and administrative operations.
Document dashboard TLS trust rather than relying on browser bypasses.
Define report retention and cleanup.
Add operational metrics:
events fetched
events processed
failures

model latency
malformed responses
human-review rate
analyst-confirmed false positives
Phase 5 — Meaningful SOC functionality: 4–8 weeks
Only after Phases 1–3 are stable:
Correlate related alerts by host, user, source IP, and time window.
Deduplicate repeated low-value events.
Add an analyst review queue.
Add case export or integration with an incident-management platform if needed.
Build a sanitized evaluation dataset for Qwen.
Measure hallucination, over-escalation, under-escalation, and recommendation quality.
Add dashboards for pipeline health and analyst outcomes.
Consider read-only enrichment from threat-intelligence sources.
Do not add autonomous blocking, host isolation, or command execution until deterministic safety gates, human approval, audit logging, and rollback controls exist.

Bottom line

The repository demonstrates a solid conceptual foundation: private alert retrieval, cautious local-model use, TLS verification, and clear human-review boundaries. Its biggest problem is not model selection—it is pipeline trustworthiness.

The immediate order of work should be:
Remove sensitive public artifacts and verify LM Studio is loopback-only.
Replace the 20-alert batch window with checkpointed pagination.
Add a strict AI output contract with fail-closed human review.
Make the read-only OpenSearch role reproducible and testable.
Add automated tests, CI, dependency pinning, and a license.

After those changes, the project can credibly evolve from a manually executed AI report generator into a dependable SOC analyst assistant.